

Developer Handoff

Current repository: `E:\vardhnam ad images\vardhnam agro app`

The project is currently a **Phase 4E foundation**, not a fully production-ready marketplace. The main documentation confirms this in [README.md (line 3)](/E:/vardhnam ad images/vardhnam agro app/README.md:3) and the current boundary is documented in [README.md (line 86)](/E:/vardhnam ad images/vardhnam agro app/README.md:86).

Completed So Far

1. Monorepo foundation

- Root workspace with:
 - `apps/marketplace-api`
 - `apps/business-web`
 - `apps/farmer-mobile`
 - `apps/partner-mobile`
 - `packages/shared-types`
 - `packages/validation`
 - `packages/api-client`
 - `packages/design-tokens`
- Root scripts exist for build, lint, test, typecheck, Prisma generation, migration, and seed.
- Docker compose is defined for local PostgreSQL and Redis.

2. Backend API foundation

- NestJS API under `/api/v1`.
- Prisma/PostgreSQL schema and migrations exist through Phase 4E.
- Redis readiness support exists.
- OpenAPI setup exists.
- Standard response envelope, error envelope, validation, correlation IDs, JSON logging, and audit service are present.
- Backend modules implemented include identity, access, organisations, onboarding, catalogue, inventory, offers, marketplace discovery, farmers, cart, checkout, payments, fulfilment orders, and audit.

3. Access and onboarding

- Users, organisations, memberships, roles, permissions.
- Mock-auth guard for development.
- Company/distributor onboarding profiles.
- KYC metadata lifecycle.
- Organisation approval/rejection with audit logs.
- Business portal views for onboarding queues and detail review.

4. Catalogue and inventory

- Company-owned brands, master products, variants, and product document metadata.
- Catalogue submission/review/approval flows.
- Distributor warehouses.
- Batch inventory.

- Append-only inventory movements.
 - Inventory ageing, low-stock, and expiring-batch report foundations.
 - Business portal catalogue, inventory, warehouse, and ageing views.
- 5. Distributor offers and marketplace discovery**
- Distributor-owned offers linked to approved products, variants, warehouses, and optional batches.
 - Offer submission, approval, pause, reactivation, and archive flows.
 - Farmer-safe public marketplace discovery API.
 - Discovery filters by pincode, category, brand, and search.
 - Availability is backend-derived from inventory movement balances.
- 6. Farmer commerce foundation**
- Farmer profiles and addresses.
 - Authenticated farmer cart API.
 - Cart item validation against approved offers, pincode serviceability, and inventory availability.
 - Idempotent checkout from cart.
 - Parent product checkout records.
 - Distributor-split child product orders.
 - Batch-level inventory reservation through append-only movements.
 - Farmer-owned order read APIs.
 - Farmer-owned cancellation for unpaid/payment-failed records with reservation release.
- 7. Mock payment foundation**
- Backend mock payment intents.
 - Idempotent mock payment creation and confirmation.
 - Payment event history.
 - Checkout/order status transitions for mock success/failure.
 - No real payment capture yet.
- 8. Distributor fulfilment foundation**
- Distributor fulfilment order list/detail APIs.
 - Accept/reject order transitions.
 - Ready-to-pack and packed transitions.
 - Invoice snapshot generation.
 - Dispatch readiness.
 - Delivery assignment.
 - Out-for-delivery transition.
 - Backend mock OTP delivery completion.
 - Audit logs for fulfilment and delivery actions.
- 9. Business web portal**
- Next.js portal connected server-side to backend APIs.
 - Pages exist for:
 - onboarding
 - catalogue
 - inventory
 - inventory ageing
 - offers
 - orders
 - order detail fulfilment actions

- audit logs
 - Mock-auth headers are sent from server-side code only.

10. Mobile apps

- Farmer Flutter app skeleton exists.
- Farmer app has API-backed marketplace browsing, plus preview/skeleton screens for cart, checkout, mock payment, and cancellation.
- Partner Flutter app skeleton exists with role-surface scaffolding.
- Hindi/English localization files exist in both mobile apps.

Verification Run

These passed:

- `npm.cmd test`
 - 16 backend unit test suites passed.
 - 73 tests passed.
- `npm.cmd run typecheck`
 - API, web, and shared packages passed.
- `npm.cmd run lint`
 - API, web, and shared packages passed.
- `npm.cmd run build`
 - NestJS API built successfully.
 - Next.js business web built successfully.
 - Shared packages built successfully.

These could not be fully verified:

- `npm.cmd --workspace @vardhnam/marketplace-api run test:integration`
 - Failed because `DATABASE_URL` and `REDIS_URL` were not set.
 - Docker is not available on PATH on this machine, so local Postgres/Redis could not be started from here.
- Flutter tests were not run.
 - `flutter` is not available on PATH.

Important Bugs / Gaps

- Git repository is not ready for normal developer workflow**
 - All files are currently untracked.
 - `git status` also reports dubious ownership unless run with a safe-directory override.
 - Developer should fix repo ownership/safe-directory and make an initial reviewed commit.
- Integration tests require environment setup**
 - `.env.example` contains required values:
 - `DATABASE_URL`
 - `REDIS_URL`

- Integration tests will fail until Postgres and Redis are running and env vars are loaded.
 - This is an environment/setup blocker, not yet a confirmed application logic bug.
3. **Production authentication is not implemented**
- Current protected APIs use mock auth.
 - `AUTH_MODE=otp` exists in env schema, but production OTP/login/session flow is not implemented.
 - This blocks real farmer, business, and partner app usage.
4. **Real integrations are missing**
- No real SMS/WhatsApp/email provider.
 - No real payment provider.
 - No production webhook signature verification.
 - No Tally write-back.
 - No invoice PDF generation.
 - No private file storage for KYC/product documents.
5. **Finance is not implemented**
- No immutable financial ledger yet.
 - No distributor payable.
 - No marketplace commission.
 - No promoter commission.
 - No delivery payout.
 - No settlements.
 - No refund ledger.
 - This is explicitly the next major roadmap phase in [docs/DEVELOPMENT_ROADMAP.md (line 100)](/E:/vardhnam ad images/vardhnam agro app/docs/DEVELOPMENT_ROADMAP.md:100).
6. **Partner network is mostly not implemented**
- Partner app is only a skeleton.
 - Promoter workflows are missing.
 - Delivery partner workflows are missing from mobile.
 - Service provider workflows are missing.
 - Service listings and service bookings are missing.
 - This is Phase 6 in [docs/DEVELOPMENT_ROADMAP.md (line 109)](/E:/vardhnam ad images/vardhnam agro app/docs/DEVELOPMENT_ROADMAP.md:109).
7. **Farmer mobile is incomplete**
- Marketplace browsing is API-backed.
 - Cart, checkout, payment, order detail, cancellation, invoice download, and delivery tracking still need full API wiring.
 - It should not calculate price, payment state, availability, or inventory locally.
8. **Service orders are not implemented**
- Product requirements require separate `PRODUCT_ORDER` and `SERVICE_ORDER` workflows.
 - Current implementation is product-order focused.
9. **Returns, refunds, disputes, and support tickets are missing**
- Cancellation exists only for eligible unpaid/payment-failed flows.

- Paid-order refund and post-delivery return windows are not implemented.
10. **Documentation drift**
- Some docs still mention earlier phase names in places while the repo is now Phase 4E.
 - Example: backend Swagger description should be checked for stale “Phase 3C” wording.

Next Execution Plan

1. Stabilize development setup

- Fix Git safe-directory/ownership.
- Commit current foundation as the initial baseline.
- Install/verify Docker Desktop.
- Copy `.env.example` to `.env`.
- Start Postgres and Redis with `docker compose up -d postgres redis`.
- Run:
 - `npm run prisma:generate`
 - `npm run prisma:migrate`
 - `npm run seed`
 - `npm test`
 - `npm run typecheck`
 - `npm run lint`
 - `npm run build`
 - `npm --workspace @vardhnam/marketplace-api run test:integration`

2. Finish production authentication

- Implement OTP auth for farmers.
- Implement business-user login/session handling.
- Replace mock-auth dependency in real environments.
- Add rate limits for OTP/login.
- Add tests for auth, session expiry, role checks, and resource ownership.

3. Generate and enforce shared API contracts

- Generate typed clients from OpenAPI.
- Replace manual duplicated API types where possible.
- Use the generated client in business web and mobile integration layers.
- Keep `/api/v1` versioning.

4. Complete farmer mobile

- Wire login/OTP.
- Wire product detail.
- Wire cart add/update/remove.
- Wire checkout from cart with idempotency key.
- Wire mock/sandbox payment.
- Wire order list/detail.
- Wire cancellation eligibility.
- Wire invoice download once PDFs exist.

- Add low-bandwidth states, Hindi/English copy, loading/error/empty states, and widget/integration tests.
- 5. Complete partner mobile**
 - Add role-based login.
 - Promoter: farmer registration assist, product sharing, assisted order attribution.
 - Delivery partner: assigned deliveries, pickup, out-for-delivery, OTP completion, proof capture.
 - Service provider: service listing/availability/bookings.
 - Ensure promoter screens cannot see delivery earnings, and delivery partners cannot see promoter commissions.
- 6. Implement finance Phase 5**
 - Add immutable ledger tables.
 - Record farmer payment, distributor payable, marketplace commission, fulfilment fee, delivery fee, promoter commission, tax, refund, adjustment, and settlement separately.
 - Calculate only on backend using integer paise.
 - Add commission finalization after delivery and return/dispute window.
 - Add finance manager portal screens and export controls with audit logs.
- 7. Implement real payments and refunds**
 - Add payment-provider abstraction.
 - Integrate sandbox provider first.
 - Add signed webhook verification.
 - Make webhooks idempotent.
 - Add refund workflow for paid cancellations/returns.
 - Add reconciliation reports.
- 8. Implement delivery production workflows**
 - Add notification abstraction for SMS/WhatsApp/push/email.
 - Send delivery OTP through provider in sandbox first.
 - Add geotag/proof capture with explicit permission.
 - Add delivery payout calculation.
 - Add failed delivery/retry workflow.
- 9. Implement service orders**
 - Add service catalogue/listings.
 - Add service-provider availability.
 - Add service booking order type separate from product inventory.
 - Add service fulfilment lifecycle, payment, cancellation, and settlement rules.
- 10. Implement returns, disputes, and support**
 - Return request flow.
 - Dispute workflow.
 - Support tickets.
 - Refund approval.
 - Audit all decisions.
 - Protect financial/order records from physical deletion.
- 11. Finish documents and storage**
 - Add secure upload validation.
 - Store KYC/product files privately.
 - Serve via signed/authorized URLs.

- Generate invoice PDFs.
- Add GST breakup rules after finance/legal approval.

12. Final release readiness

- Add E2E tests for full MVP journey described in [docs/PRODUCT_REQUIREMENTS.md (line 289)](/E:/vardhnam ad images/vardhnam agro app/docs/PRODUCT_REQUIREMENTS.md:289).
- Add mobile tests.
- Add CI for API, web, mobile, migrations, lint, typecheck, and integration tests.
- Run security review.
- Run data migration review.
- Prepare staging deployment only after secrets and provider sandbox configs are properly managed.

Bottom line: the backend and business portal have a strong Phase 4E foundation. The next real work is turning this from a mock/local foundation into a fully functioning marketplace by completing production auth, farmer mobile API flows, partner app workflows, finance, real integrations, service orders, and post-payment operations.